SupplyGuard: Logistics & Delivery Intelligence Project

Core technologies

```
None
Python, NumPy, Pandas, Matplotlib, Jupyter Notebook, CSV
Git, GitHub.
```

The project covers the areas in your learning material: acquisition from CSV, NumPy and Pandas, inspection, cleaning, datetime handling, duplicates/outliers, merging, grouping, feature engineering, scaling, encoding, pipelines, reshaping, visualisation and exporting.

---

1. Business Scenario

A fictional South African logistics company, **SupplyGuard Logistics**, has provided customer, shipment, driver, vehicle, warehouse and delivery data.

Management wants the data science team to investigate:

> **How efficiently is the company operating, where are delivery activities concentrated, and which shipments or operational factors may represent a delivery or business risk?**

You must build a reproducible data-analysis solution and present your findings to the company's executive team.

---

2. Dataset Package

You will work from raw datasets.

```
None
logistics_data/
│
├── customers.csv
├── shipments.csv
├── deliveries.csv
├── drivers.csv
├── vehicles.csv
├── warehouses.csv
├── delivery_labels.csv
└── exchange_rates.csv
```

Relationships

```
None
customers.csv
      │
      ↓
shipments.csv
      │
      ↓
deliveries.csv
      │
      ├──────────→ drivers.csv
      │
      ├──────────→ vehicles.csv
      │
      └──────────→ delivery_labels.csv
      │
shipments.csv
      │
      ↓
warehouses.csv
```

This is a **relational data problem**. You are expected to work with multiple related datasets rather than analysing only one CSV file.

---

3. The Datasets Are Intentionally Messy

The raw datasets will contain realistic data-quality problems.

You must **discover, investigate and justify** how you handle them.

Missing values

Examples:

```
None
customer_id
driver_id
vehicle_id
delivery_date
warehouse_id
```

Duplicates

Some duplicate customers, shipments and delivery records may exist.

Inconsistent text

Examples:

```
None
Western Cape
western cape
WESTERN CAPE
```

Inconsistent delivery statuses

Examples:

```
None
Delivered
delivered
DELIVERED
Out for Delivery
out for delivery
OUT FOR DELIVERY
```

Numeric problems

Examples:

```
None
"1,500.00"
"8 250.50"
"4500"
```

Date problems

Examples:

```
None
2026-07-01
01/07/2026
2026/07/01
```

Outliers

For example:

```
None
Normal delivery distance: 15 km
Normal delivery distance: 50 km
Normal delivery distance: 120 km
Unusual delivery distance: 1,250 km
```

---

4. Data Engineering & Customer Analytics

Mission

> **Build a trusted logistics dataset and use it to understand customers, shipments and delivery performance.**

Responsibilities

4.1 Data Acquisition

Load the datasets using Pandas.

For example:

```python
pd.read_csv()
```

Inspect every dataset.

You must demonstrate:

```python
.head()
.tail()
.info()
.shape
.describe()
.dtypes
```

4.2 Data-Quality Investigation

Identify:

- missing values
- duplicates
- incorrect data types
- inconsistent text
- invalid dates
- invalid IDs
- outliers
- referential integrity problems

Produce:

```
None
data_quality_report.csv
```

4.3 Data Cleaning

You must demonstrate appropriate use of:

```python
.dropna()
.fillna()
.drop_duplicates()
.astype()
.str.strip()
.str.lower()
.str.replace()
pd.to_datetime()
```

You must document **why** you chose each treatment.

For example:

> Missing delivery distances were investigated against shipment and warehouse records before deciding whether the values should be removed, replaced or retained.

4.4 Dataset Merging

Create the trusted master dataset.

For example:

```python
master = customers.merge(
    shipments,
    on="customer_id",
    how="left"
)
```

Then:

```python
master = master.merge(
    deliveries,
    on="shipment_id",
    how="left"
)
```

Then merge drivers, vehicles and warehouses.

Final output:

```
logistics_master.csv
```

4.5 Feature Engineering

Create customer, shipment and delivery features.

Date features

```
year
month
day
weekday
hour
is_weekend
```

Customer features

```
customer_shipment_count
customer_total_shipping_value
customer_average_shipment_value
customer_average_delivery_time
```

Shipment features

```
shipment_weight
shipment_distance
delivery_duration
late_delivery_flag
```

4.6 Customer & Delivery Analytics

Answer the following questions.

Customer Questions

1. How many customers does the company have?
2. Which province has the most customers?
3. Which customer segment sends the most shipments?
4. Which customer segment generates the highest shipping value?
5. Which customers have the highest shipment frequency?
6. Which customers experience the most delayed deliveries?

Shipment Questions

7. Which shipment types are most common?
8. Which delivery methods are most frequently used?
9. Which province generates the most shipment volume?
10. What are the busiest shipping days?
11. How does shipment activity change over time?

---

5. Notebook

Create:

```
None
notebooks/
└── customer_operations.ipynb
```

Use this structure:

```
None
1. Business Problem
2. Dataset Overview
3. Data Acquisition
4. Data Inspection
5. Data Cleaning
6. Data Validation
7. Dataset Merging
8. Feature Engineering
9. Customer Analysis
10. Shipment Analysis
11. Delivery Analysis
12. Visualisations
13. Findings
14. Recommendations
```

6. Delivery Risk & Business Intelligence

Mission

> **Use the trusted logistics data to identify delayed, high-risk or inefficient deliveries and provide recommendations to management.**

Use:

```
None
logistics_master.csv
```

However, you must **validate the dataset before using it**.

6.1 Delivery Risk Analysis

Investigate:

- unusually long delivery times
- unusually long delivery distances
- repeated delivery delays
- high-risk routes
- high-risk warehouses
- vehicle-related delivery problems
- driver-related delivery patterns
- unusual delivery times
- customer behaviour associated with repeated delays

6.2 Delivery Feature Engineering

Create features such as:

```
None
delivery_hour
delivery_weekday
customer_shipment_count
customer_average_delivery_time
distance_vs_average
delivery_delay_hours
late_delivery_flag
long_distance_flag
repeat_delay_flag
```

Example:

```python
df["distance_vs_average"] = (
    df["distance_km"] / df["customer_average_distance"]
)
```

6.3 Operational Risk Scoring

Create a simple, explainable delivery-risk score.

For example:

```
Late delivery                 +25
Very long delivery distance   +15
Repeated customer delays      +20
Long delivery duration        +20
High-risk route               +20
```

Then:

```
0–30       Low Risk
31–60      Medium Risk
61–100     High Risk
```

The exact scoring rules must be documented and justified.

6.4 Delivery Performance Analysis

Use `delivery_labels.csv` to investigate:

```
Late delivery rate
Late deliveries by route
Late deliveries by driver
Late deliveries by vehicle
Late deliveries by warehouse
Late deliveries by province
Late deliveries by shipment type
Late deliveries by delivery time
```

6.5 Business Intelligence

Answer the following questions.

Operations

1. How many deliveries are late?
2. What percentage of deliveries are late?
3. Which routes have the highest late-delivery rate?
4. Which drivers have the highest delay rates?
5. Which vehicles are associated with the most delays?
6. Which warehouses have the highest operational risk?
7. What times of day show the greatest delivery delays?

Management

8. What are the three biggest operational risks?
9. What should management investigate?
10. What operational controls could reduce delivery delays?
11. Which routes, drivers, vehicles or warehouses require attention?
12. How could the company improve customer satisfaction?

7. Notebook

Create:

```
None
notebooks/
└── delivery_risk.ipynb
```

Use this structure:

```
None
1. Business Problem
2. Load Trusted Dataset
3. Data Validation
4. Exploratory Analysis
5. Risk Feature Engineering
6. Delivery Performance Analysis
7. Risk Scoring
8. Visualisations
9. Key Findings
10. Recommendations
```

8. GitHub Workflow

Use **one repository for the whole team**.

```
None
supplyguard/
│
├── data/
│   ├── raw/
│   ├── processed/
│   └── final/
│
├── notebooks/
│   ├── customer_operations.ipynb
│   └── delivery_risk.ipynb
│
├── requirements.txt
└── README.md
```

9. Branching Strategy

Use:

```
None
main
│
├── data-engineering
│
├── customer-analysis
│
├── delivery-risk
│
└── business-intelligence
```

Each member must have an identifiable contribution.

Examples:

```
Shell
git add .
git commit -m "feat: clean delivery distances"
git push
```

Another:

```
Shell
git commit -m "feat: add customer shipment analysis"
```

Another:

```shell
git commit -m "feat: calculate delivery risk score"
```

---

10. Pull-Request Requirement

Each member must:

1. Create a branch.
2. Push changes.
3. Open a Pull Request.
4. Have another member review it.
5. Receive feedback.
6. Make changes where necessary.
7. Merge the Pull Request.

GitHub is part of the **assessment**. Do not simply upload your completed work at the end.

11. Roles Within the Team

Data Engineering

Responsible for:

- CSV acquisition
- cleaning
- validation
- merging
- pipeline

Data Analytics

Responsible for:

- Pandas
- feature engineering
- analysis
- statistics

Data Science / Storytelling

Responsible for:

- visualisation
- modelling/risk
- findings
- recommendations
- presentation

SPRINT PLAN

Build the Data Foundation

Phase 1 — Project Launch

Complete:

- Understand the business problem.
- Explore the datasets.
- Create the GitHub repository.
- Clone the repository.
- Create branches.
- Assign roles.
- Create a data dictionary.

Phase 2 — Data Acquisition & Inspection

Task 1

Focus on:

```
None
CSV acquisition
Data inspection
Relationships
Data types
Data quality
```

Task 2

Focus on:

```
None
Independent exploratory analysis
Delivery labels
Risk questions
Business questions
```

Phase 3 — Cleaning

Focus on:

```
None
Missing values
Duplicates
Dates
Strings
Numeric values
Outliers
Invalid records
```

All cleaning decisions must be documented.

Phase 4 — Integration

Combine:

```
None
customers
     +
shipments
     +
deliveries
     +
drivers
     +
vehicles
     +
warehouses
```

Produce:

```
None
logistics_master.csv
```

Also validate:

```
None
Are the joins correct?
Are shipment counts preserved?
Are IDs valid?
Are there unexpected duplicates?
```

Phase 5 — Feature Engineering

Work with the integrated data.

Customer and operational features:

```
None
customer features
shipment features
delivery features
date features
```

Risk features:

```
None
delivery risk features
delay indicators
route indicators
```

Checkpoint

Give a **short progress demo**.

Show:

- GitHub
- Notebook
- cleaned data
- analysis
- features
- preliminary findings

Insights & Intelligence

Phase 6 — Analysis

Task 1

> Customer, shipment and delivery analytics

Task 2

> Delivery risk and operational analytics

Phase 7 — Visualisation

Create:

- bar chart
- line chart
- pie chart
- histogram
- scatter plot

Integration & Code Review

The team must:

- review code
- review notebooks
- open Pull Requests
- resolve issues
- update README
- clean the GitHub repository
- prepare the presentation

Your final solution must provide evidence-based answers to:

1. Who are our customers?

2. How are customers using our logistics services?

3. What are the major shipment and delivery patterns?

4. What unusual or inefficient delivery behaviour exists?

5. Which deliveries present the greatest operational risk?

6. What factors are associated with late deliveries?

7. What should SupplyGuard Logistics management do based on your findings?

Your recommendations must be supported by **data, analysis and visualisations**.

Final Instructions

Before submitting, make sure that:

- Your code runs without errors.
- Your notebook is organised according to the required structure.
- Your data-cleaning decisions are explained.
- Your datasets have been validated.
- Your joins and merges are correct.
- Your feature engineering is clearly explained.
- Your required business questions have been answered.
- Your visualisations support your findings.
- You completed the delivery-risk analysis.
- Your GitHub commits clearly show individual contributions.
- A Pull Request has been created and reviewed.
- Your README is complete.

- Your final outputs are saved in the correct folders.
- Every team member can explain the work they contributed.
- Every team member is prepared for the technical Q&A.
- Your presentation clearly explains what SupplyGuard Logistics should do based on the evidence.

Your goal is not simply to produce code. Your goal is to use data to solve a real business problem and communicate the solution professionally.